

# CRT Science Customization Bundle

The machine-optimized root, `CS_README.machine.md`, is authoritative; this is its human-readable twin.

A portable bundle of **generalizable** Claude Science customizations for scientific research, coding, and modeling — skills, agent profiles, and kernel gates — captured from a working research setup and scrubbed of everything project-specific. Run one installer in a Claude Science repl cell and the account inherits the same methodology, skills, and agent roster. This is the Claude **Science** twin of the Claude Research Toolkit (the Claude **Code** carrier); the two share most of their methodology content, re-expressed in each platform's own primitives.

For a day-one walkthrough, see `CS_QUICKSTART`; for the full reference, see `CS_USAGE_DETAILED`.

## What's included

Counts recomputed from `crt_science_bundle.json` on 2026-08-02: **52 skills / 18 profiles** (version 2.11).

- **Skills (52)** auto-load through `host.skills`. Some carry a `kernel.py` **sidecar** of plain callables (no CLI, no `__main__`). They fall into groups: *workflow / agency* (plan, collab, solo, supervisory-workflow, and more), *domain method* (mgcv-temporal-gam, brms-hierarchical-fitting, gap-fill-imputation, and more), and *toolkit-builder* (bash-hook-contract, toolkit-extension-authoring, software-craft, and more).
- **Profiles (18)** are agent personas dispatched through `host.delegate`: GENERALIST (the daily driver, with full reach) plus 17 named specialists (PLANNER, SOFTWARE\_DEVELOPER, RESEARCH\_STATS\_ADVISOR, LLM\_DOC\_ARCHITECT, the domain modelers, and the rest).
- **Kernel gates** are fail-closed routing and verification gates carried as skill sidecars: delegation-planning (model-route and plan-lint), directing-execution (confirm-before-stop), provenance-guard (verify-before-assert), and verification-loop (require-receipt).

The complete roster — every skill and profile, one line each — lives in `CS_USAGE_DETAILED`, which is the single source for it (it is not duplicated here, to avoid drift).

## Install

The mechanics and the paste-ready block live in `CS_INSTALL_STARTER_v2.11.md`; this is the short version. Upload `crt_science_bundle.json` and `install crt_science.py` into the Science project, then run this in a repl cell:

```
exec(open("install crt_science.py").read()); install crt_science()
```

That is the non-destructive default; pass `overwrite=True` to update an item that exists but **differs**. `host` is pre-injected in the repl kernel. The installer is idempotent: it creates an item that is absent, treats a byte-identical item as a no-op, and skips-and-reports an item that exists but differs unless `overwrite=True`. The full paste-ready block and its acceptance probes are in `CS_INSTALL_STARTER_v2.11.md`.

## Post-install verification

Summarized from steps 3–5 of CS\_INSTALL\_STARTER:

1. **Presence.** The installer reports 52 skills / 18 profiles updated or created, with no `InstallVerificationError` — a content-hash manifest re-reads the live account and fails closed on any residual drift.
2. **Kernel-gate smoke** (host-independent, so it must pass identically to the dev-side fixtures): `model_route_gate` fails on `claude-opus-5` and passes on a valid tier map; `confirm_before_stop` fails with no receipts; `verify_before_assert` fails on a memory-only source; `require_receipt` fails with no receipt.
3. **Live routing.** One `host.delegate` probe on a T4 model (`claude-haiku-4-5`) runs on the requested model. Never dispatch any probe to `claude-opus-5` — the ban applies to probes too.
4. **Sidecar acceptance.** The four sidecar skills (`delegation-planning`, `directing-execution`, `provenance-guard`, `verification-loop`) re-probe the publish gate and return `ok=True`.

## Layout

```
CSRTB/
├─ bundle_src/                                source of truth (edit here, then rebuild)
│   └─ skills/      (52)                     one dir per skill: SKILL.md (+ optional
kernel.py sidecar)
│       └─ profiles/  (18)                   one JSON per agent profile
├─ crt_science_bundle.json                   built artifact (build_crt_science_bundle.py
from bundle_src) – never hand-edit
├─ build_config.json                         bundle_version (authoritative) plus build
metadata
├─ VERSION                                  the bundle version, mirroring
build_config.json's bundle_version
├─ install_crt_science.py                   the installer (run in a Science repl cell)
├─ check_bundle_parity.py, check_sidecar_contract.py, check_currency.py  ship/
verify gates
├─ CS_INSTALL_STARTER_v2.11.md               paste-ready user-run install and acceptance
probes
└─ docs/{machine_md, human_md}/              this doc set (CS_README, CS_QUICKSTART,
CS_USAGE_DETAILED, CS_ADVANCED)
```

## The doc set — where to go next

- **CS\_README** (this document) is the orientation front door and points you to the rest.
- **CS\_QUICKSTART** is the day-one recipe for a new Science user.
- **CS\_USAGE\_DETAILED** is the full reference, including the complete 52/18 roster.
- **CS\_ADVANCED** covers the architecture (profiles, skills, kernels), the `host.*` API, memory-as-poison-surface, and extension authoring.
- **CS\_INSTALL\_STARTER\_v2.11.md** (at the bundle root) is the paste-ready user-run install and acceptance probes.

## **Uninstall / replay**

There is no `uninstall.sh` — Claude Science installs into the account, not into a file tree. Re-running `install_crt_science.py` is safe: byte-identical items are no-ops, and an item that exists but **differs** is skipped and reported unless `overwrite=True`, so a re-run never silently clobbers the account's own same-named item. You can re-check the live account against the bundle at any time with `verify_against_manifest(bundle_path)`.

## **Not included (deliberately)**

Anything tied to a specific research project — domain physics, site or data facts, project pipelines, per-project rules. This is the reusable methodology layer only.